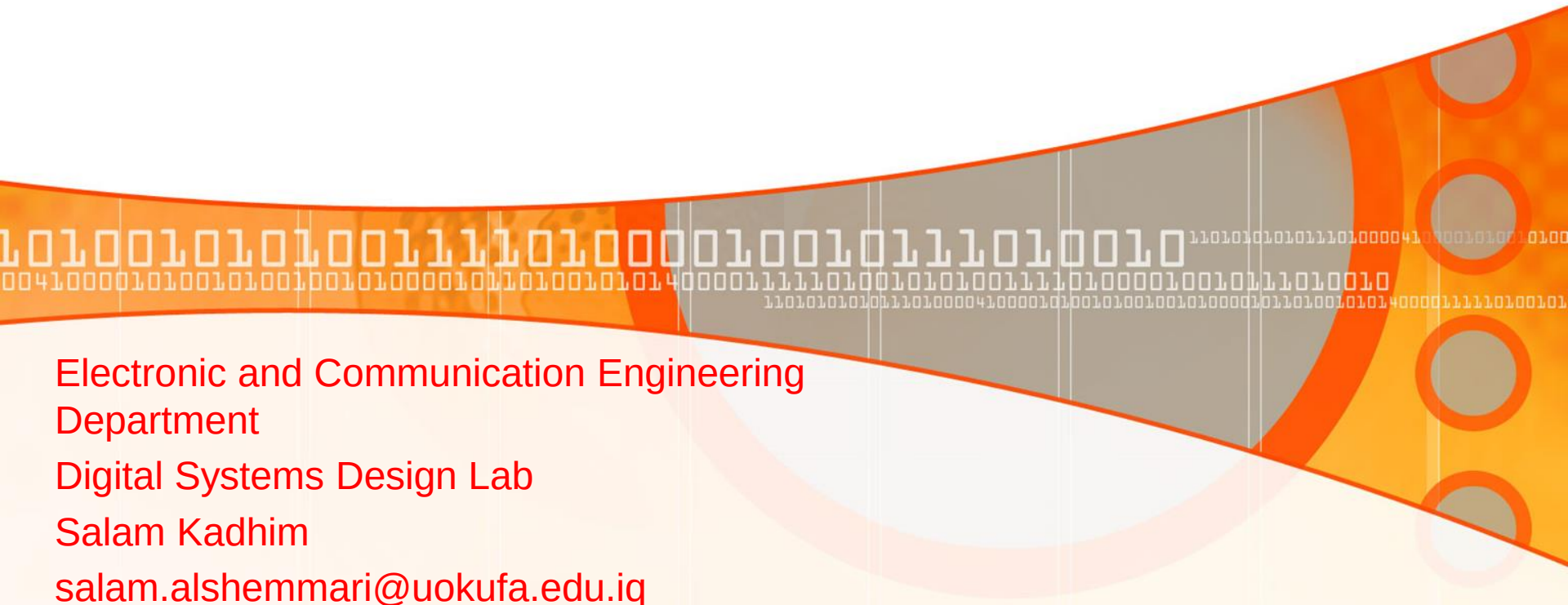


# Digital Systems Design with VHDL

(a review)

## *Lecture 1*



Electronic and Communication Engineering  
Department

Digital Systems Design Lab

Salam Kadhim

[salam.alshemmari@uokufa.edu.iq](mailto:salam.alshemmari@uokufa.edu.iq)

# Digital circuit design

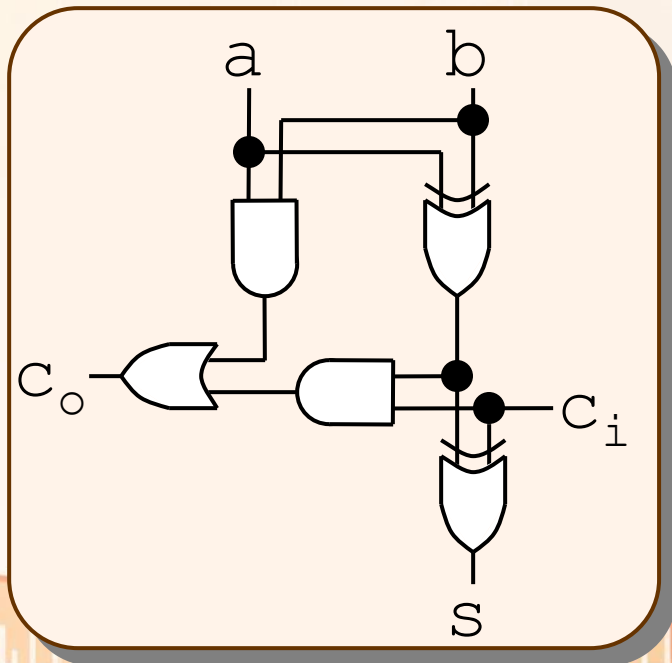
- *Module learning outcomes:*
  - *Understand and have practice in the design of complex digital circuits using a hierarchical approach*
  - *Be able to*
    - *design digital circuits using VHDL*
    - *use advanced simulation techniques to verify the operation of digital circuits*
    - *implement digital circuits in FPGA devices*
  - *Be familiar with the modern design flow for digital circuits and the relative software tools*
- *VHDL is just a tool! Circuit design is what matters – now and in any job!*

# Designing a circuit

A design project starts *on paper!*

3 input bits  $a, b, c_i$

2 outputs: sum  $s = (a+b+c_i) \bmod 2$   
$$= a\bar{b}\bar{c}_i + \bar{a}b\bar{c}_i + \bar{a}\bar{b}c_i + abc_i$$
  
carry  $c_o = (a+b+c_i) \div 2$   
$$= ab + ac_i + bc_i$$

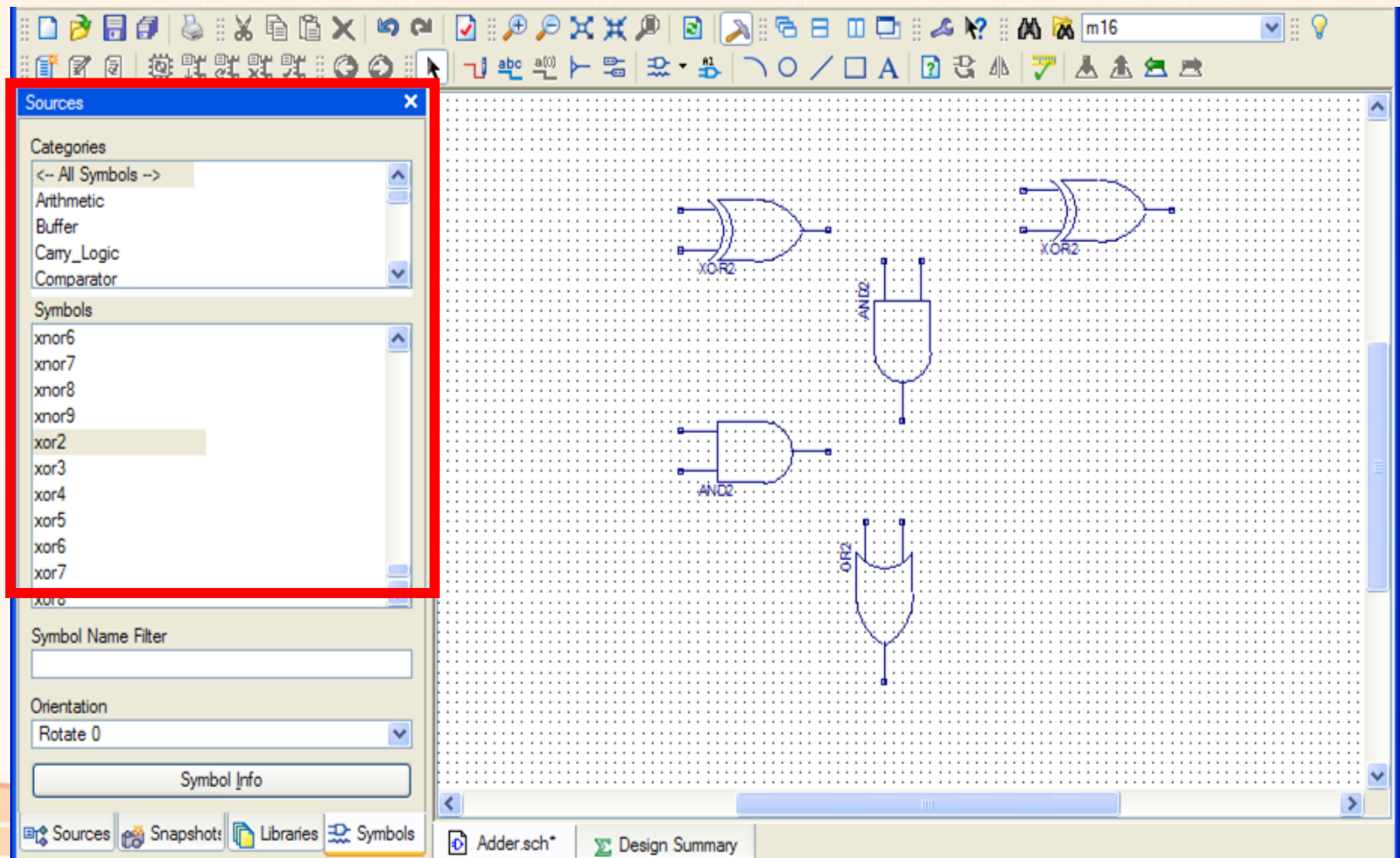


|       |   |    |    |    |    |
|-------|---|----|----|----|----|
| s:    |   | ab |    |    |    |
|       |   | 00 | 01 | 11 | 10 |
| $c_i$ | 0 | 0  | 1  | 0  | 1  |
|       | 1 | 1  | 0  | 1  | 0  |

|         |   |    |    |    |    |
|---------|---|----|----|----|----|
| $c_o$ : |   | ab |    |    |    |
|         |   | 00 | 01 | 11 | 10 |
| $c$     | 0 | 0  | 0  | 1  | 0  |
|         | 1 | 0  | 1  | 1  | 1  |

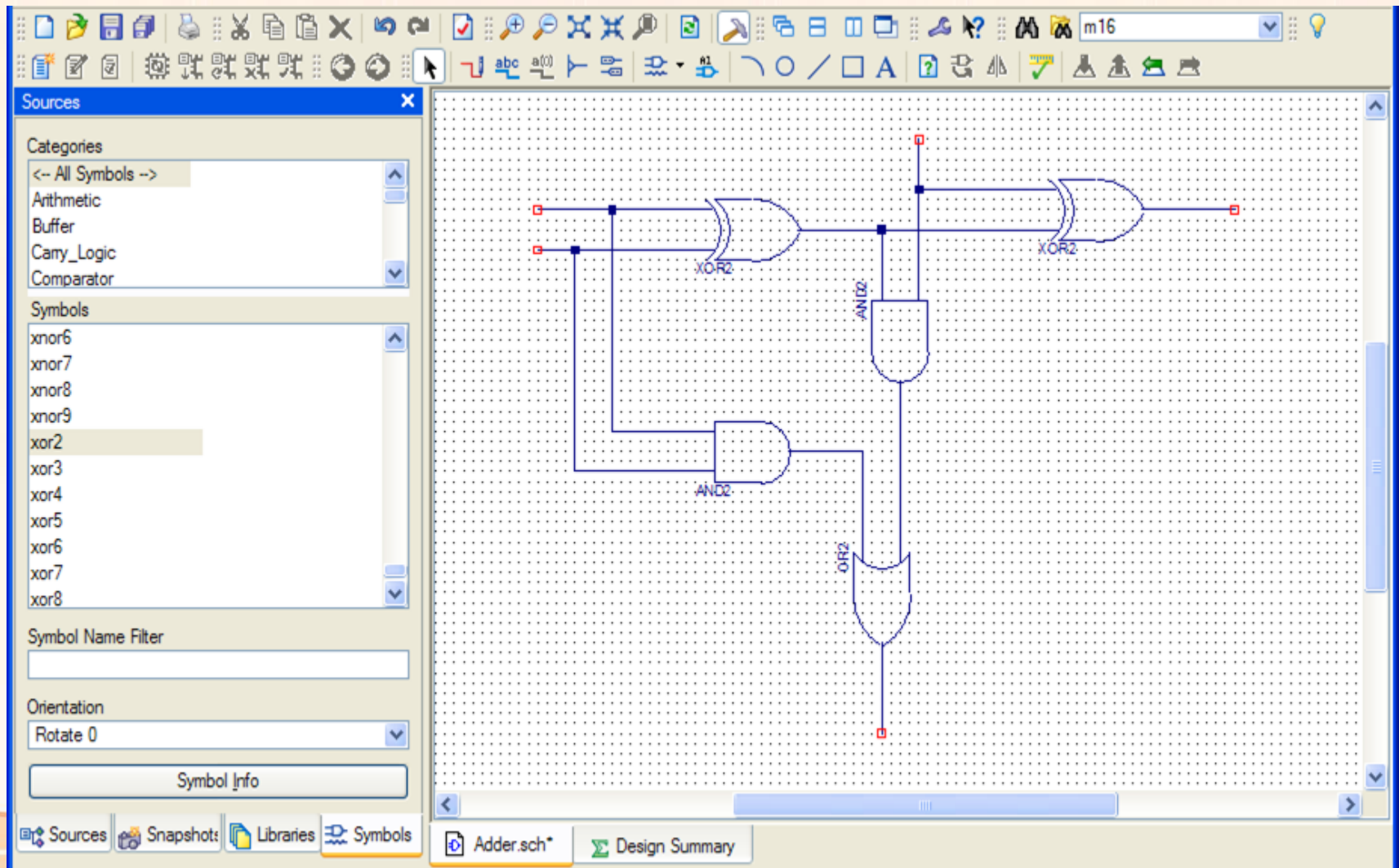
# Schematics

- Until a few years ago, the standard digital logic design entry was through **schematics**.
- Use library of components – drag&drop gates



# Schematics

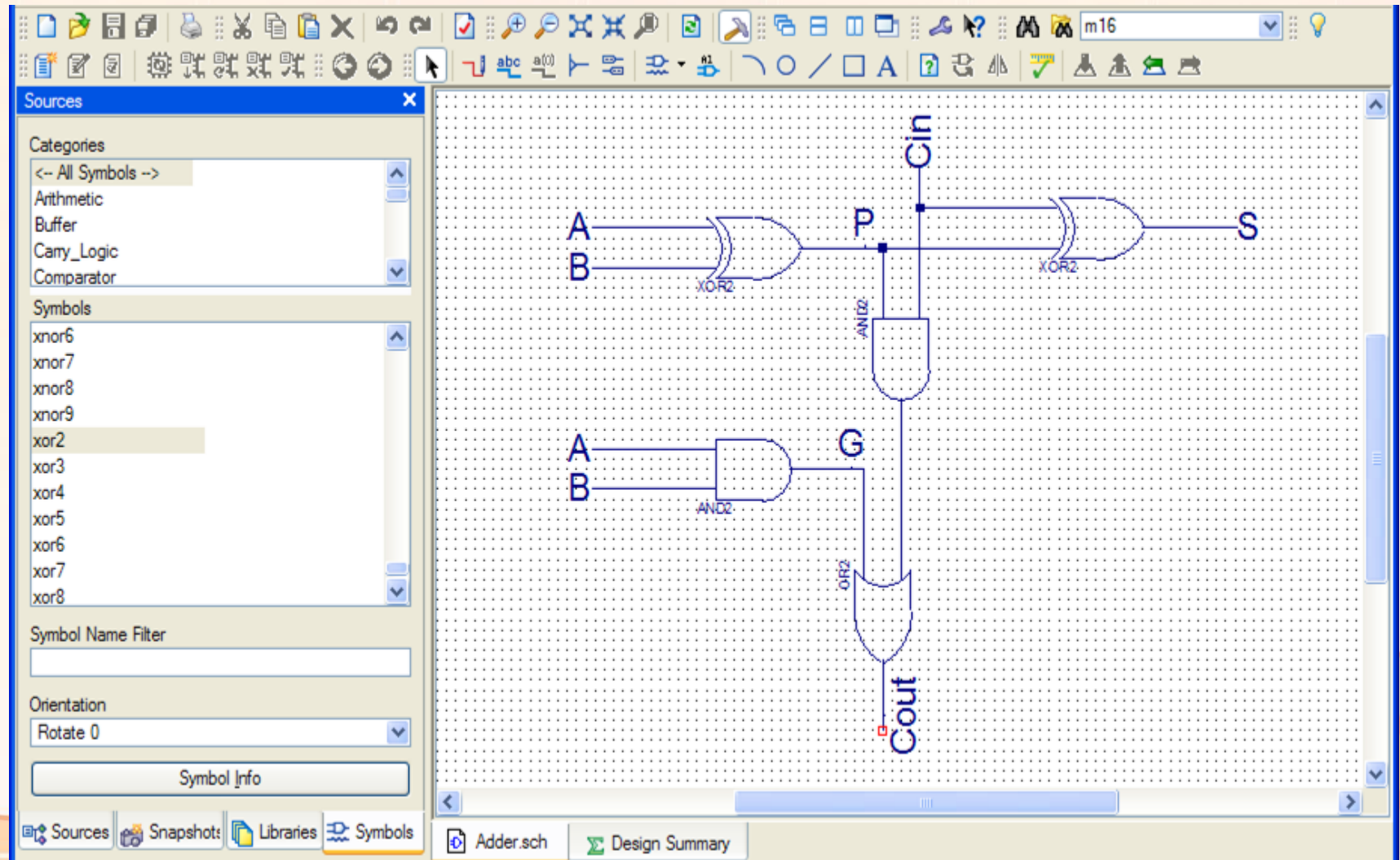
- *Connect gates together with lines*





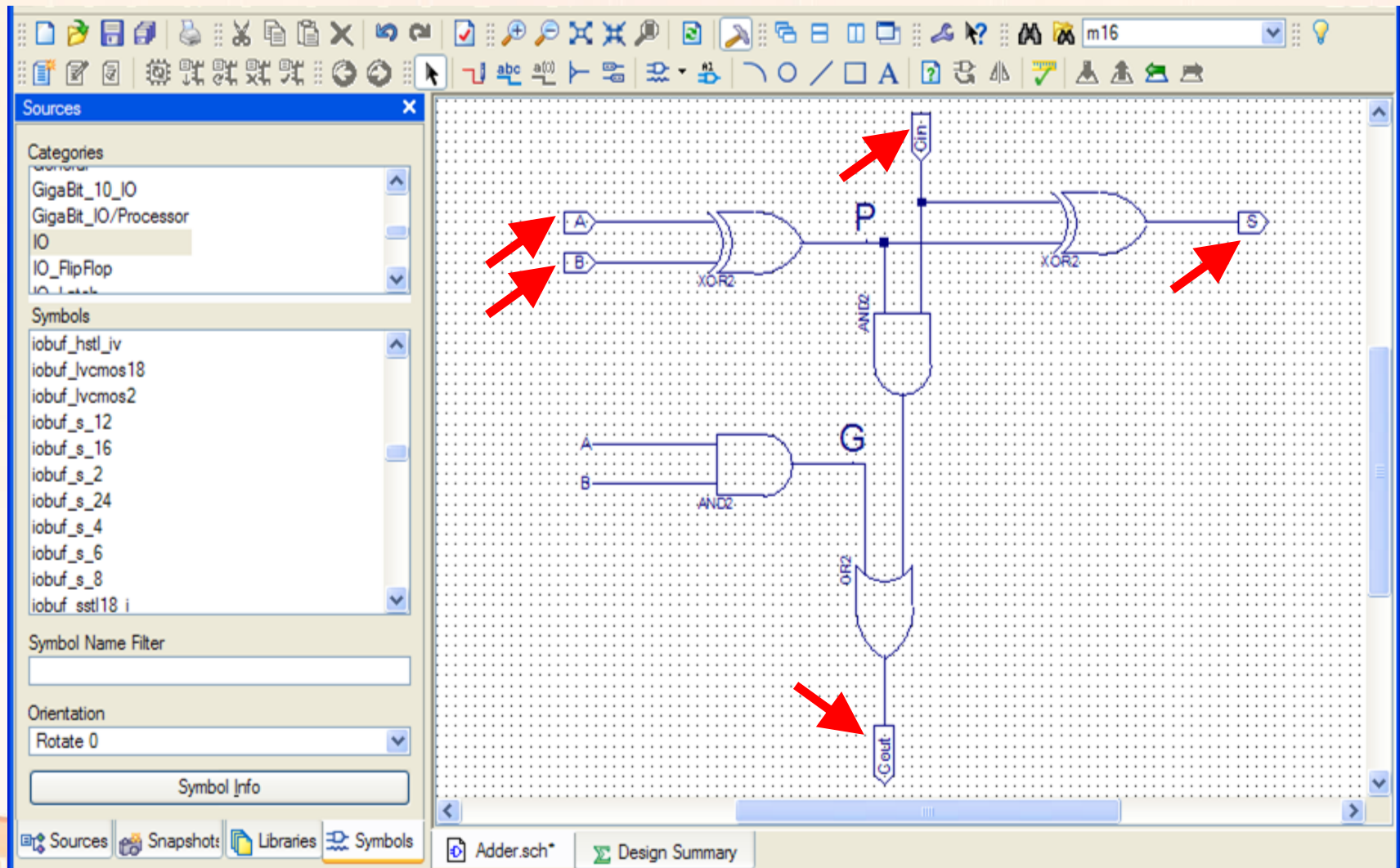
# Schematics

- *Label all “important” lines*



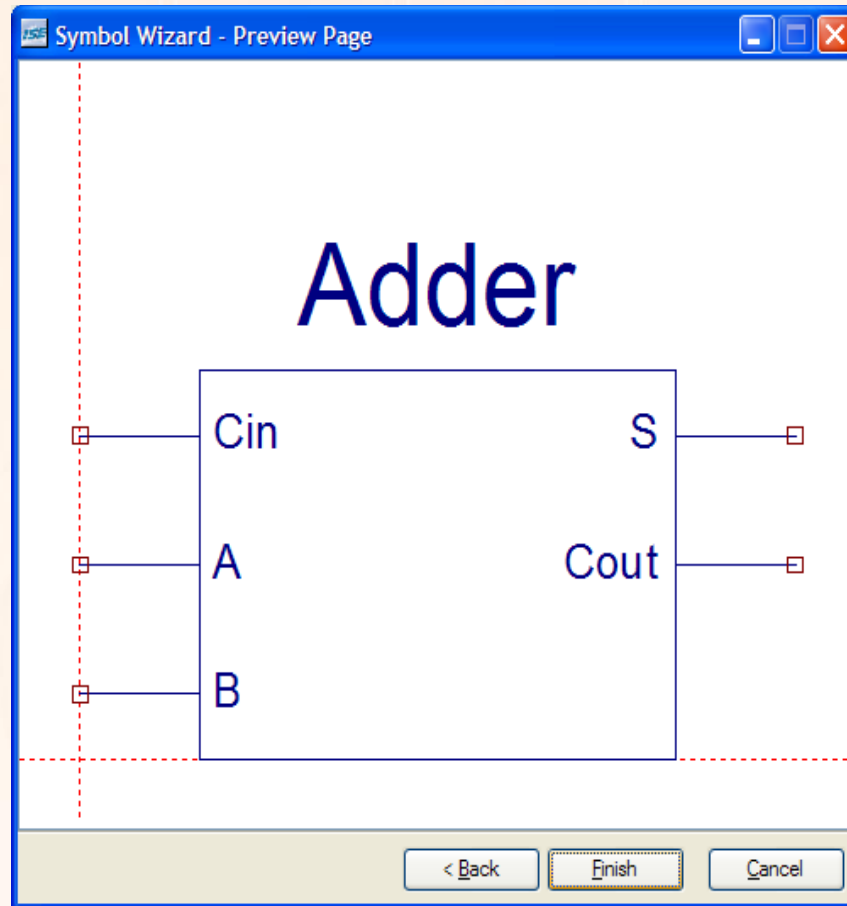
# Schematics

- *Define I/Os*



# Schematics

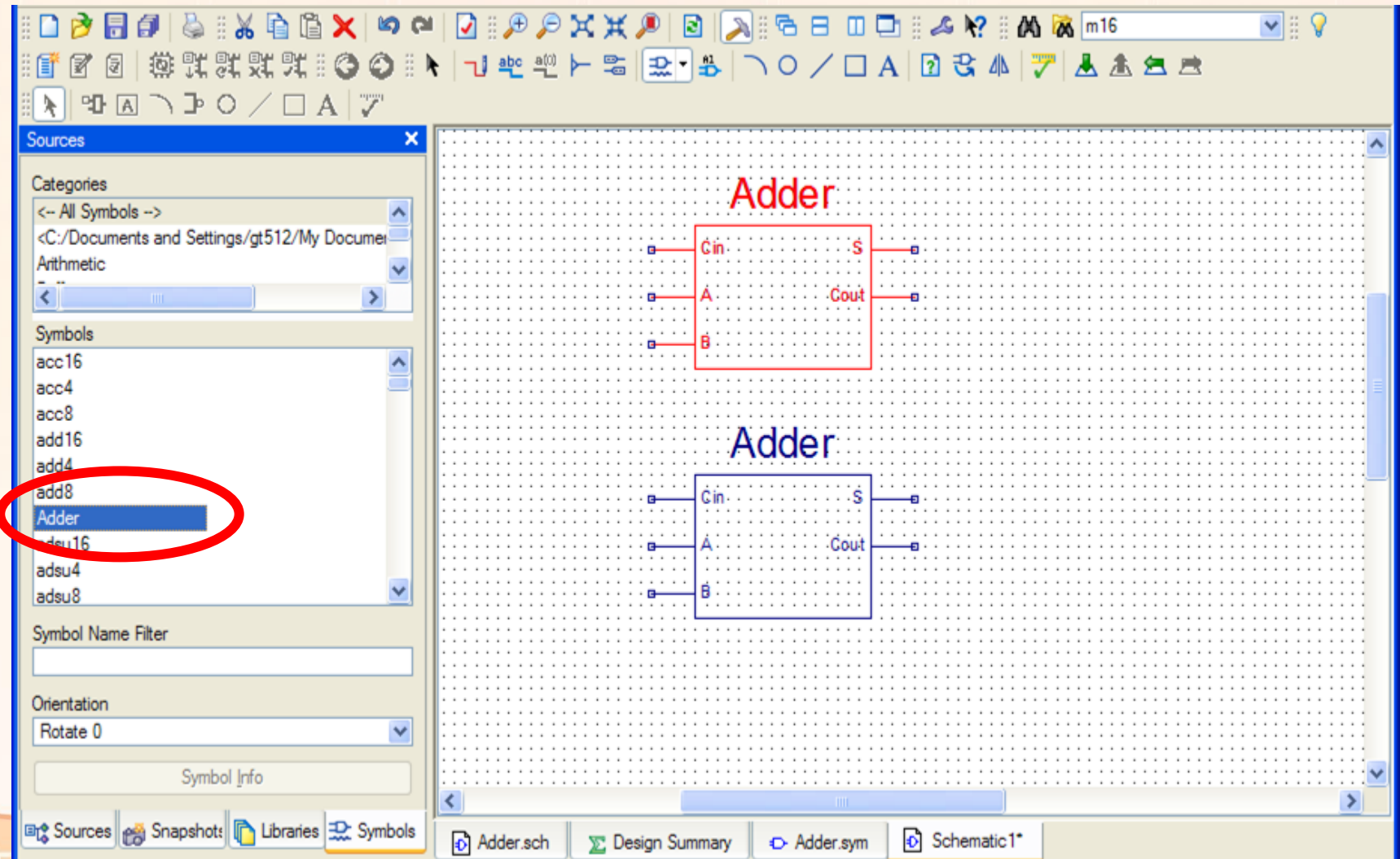
- Create a *component* out of the schematic





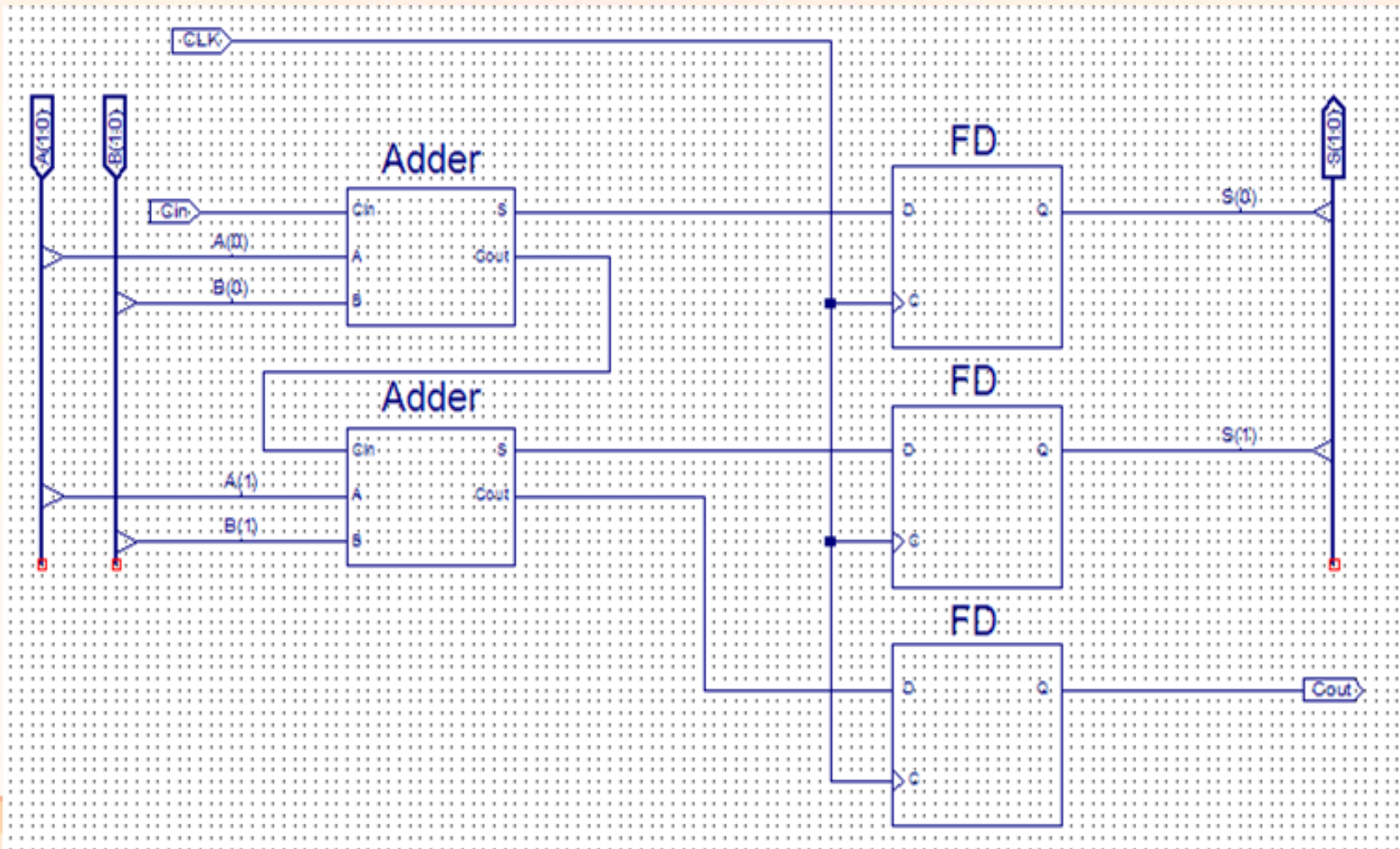
# Schematics

- The new component becomes **part of the library***



# Schematics

- Components are then used in higher-level circuits to instantiate a **hierarchical design**



# Design rules

- *Formal rules of Boolean logic always apply, but most of the methodologies you studied in the first year (Karnaugh maps, state minimization) either stop working or become irrelevant in **complex designs**.*
- *This is NOT a good thing!*
- *These are replaced by some “good design” rules that are considerably fuzzier*
  - *Hierarchical design*
  - *Component reuse*
  - *Speed (critical path analysis)*
  - *Etc.*
- *Components change as well: from single gates to **arrays of gates**, from single flip-flops to **registers**, from single lines to **busses**.*

# HDLs

- *The transition to complex designs makes schematic entry more “annoying” (bus labels, large numbers of gates, etc.) – particularly when designs have to be modified and for simulation.*
- *This has caused a shift from schematic entry to **Hardware Description Languages** – of which VHDL is one of the most widespread.*
  - *Developed in the early 1980’s as a method of describing electronic systems for the American Department of Defense*
  - *VHDL was developed to **model and simulate** digital systems. Only later it was adapted for digital synthesis.*
  - *Separate synthesis tools must be used to create and optimise the implementation*

# Hardware Description Languages

- What HDLs are: languages used for the **formal description of electronic circuits**. They can describe a circuit's operation, its design and organization (but usually not its layout), and can be used to verify its operation through simulation.
- What HDLs are NOT: **programming languages**.
- There are two main HDLs out there: Verilog (1985) and **VHDL** (**V**ery **H**igh Speed Integrated Circuit **H**ardware **D**escription **L**anguage, 1987/93/2003/(08)).
- For **system-level design**, where a higher level of abstraction is useful and design implies **both hardware and software**, other “HDLs” have been developed to combine high level languages with concurrency models and allow faster design cycles : Catapult C, Impulse C, SystemVerilog, SystemVHDL, Handel-C, ...



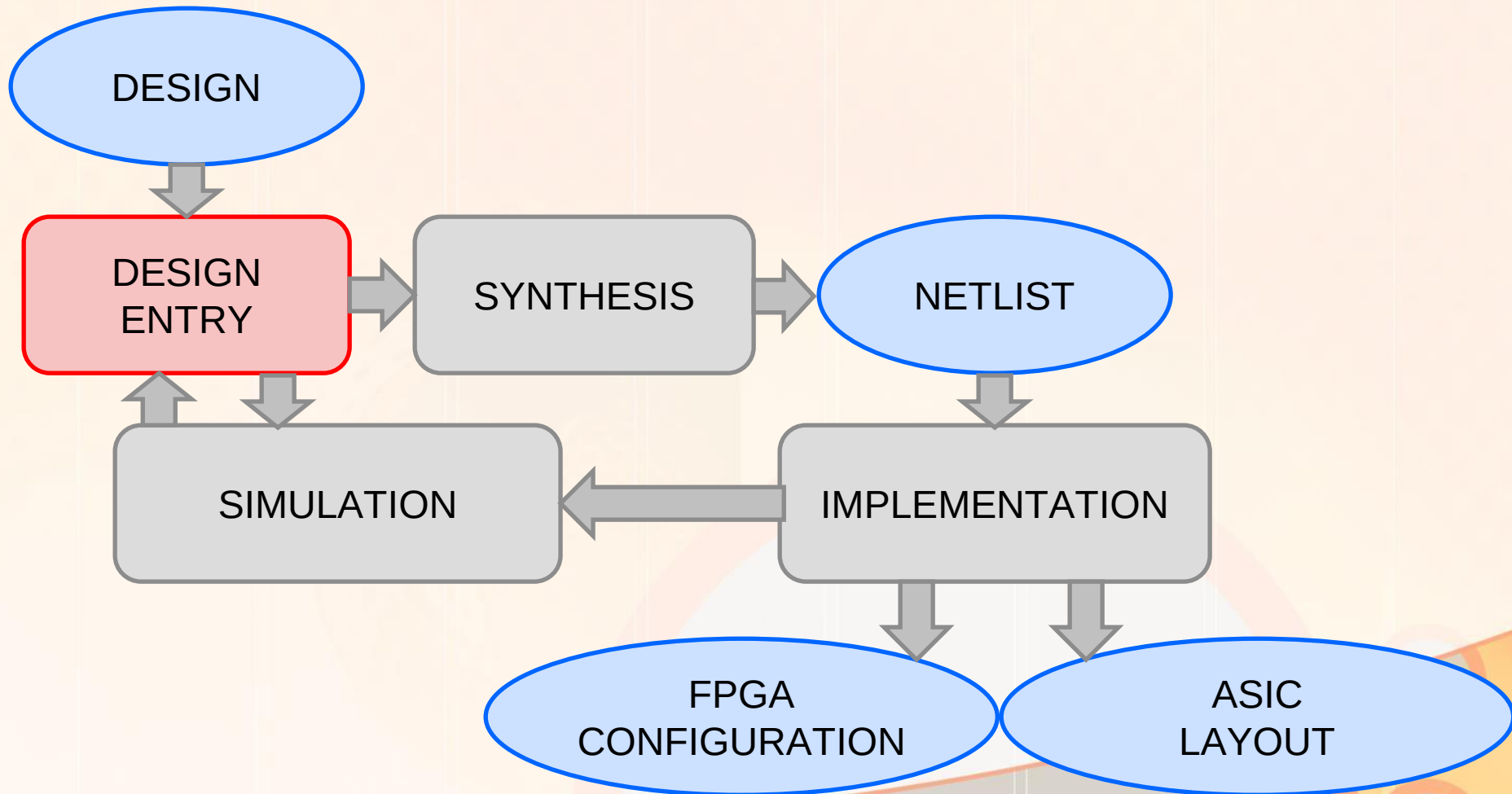
# Synthesizable VHDL

- The objective of this module is to *design digital circuits using VHDL*
  - while VHDL is (too) often used to model circuits, we are interested in generating circuits that eventually will have to be implemented (into an FPGA).
- In other words, we are interested in *synthesizable VHDL*.
- This implies that we will concentrate mostly on a *subset* of all VHDL constructs and operators. Others will be mentioned (particularly when dealing with testbenches) but the focus will be on VHDL that can be used to synthesize hardware.
- Keep that in mind if you use web-based resources!



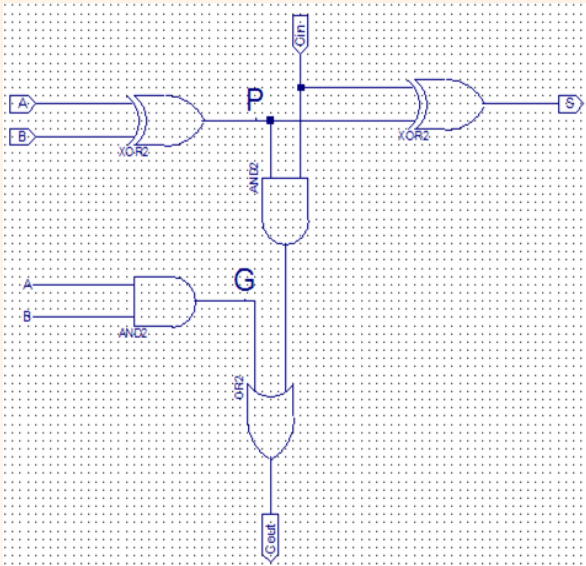
# Where HDLs fit

- HDLs replace schematics as a **design entry** tool*



# Design synthesis

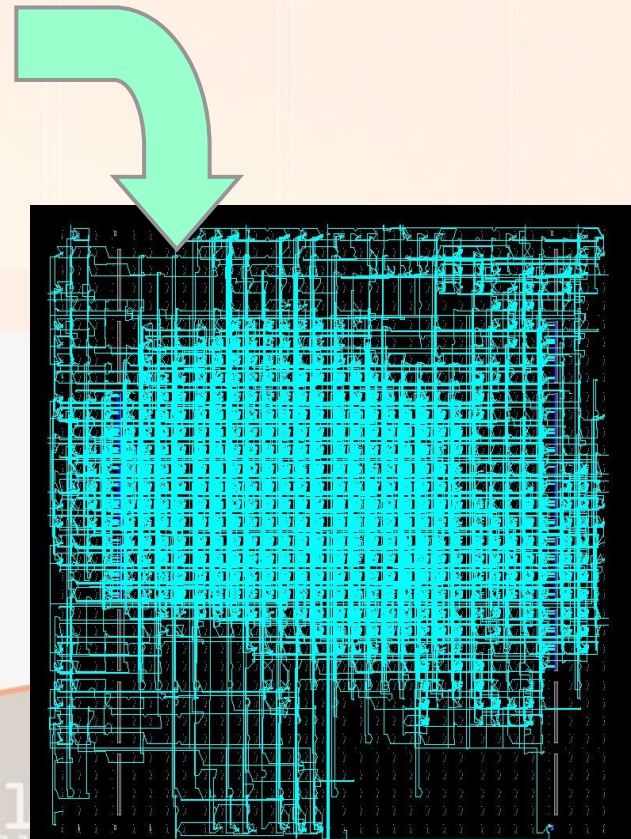
- *There is a close correlation between schematics and HDLs*
- *In practice, modern synthesis tools transform a schematic into HDL, then HDL into a configuration/layout*
- *Particularly as a beginner, you should always have a circuit schematic in mind when coding in VHDL*



```
entity ADDER is
  port (A,B: in std
        Cin: in std
        S: out std_
        Cout: out s
end ADDER;
```

```
architecture struct
  signal P,G,P1: std
begin
```

```
...
...
...
...
...
```

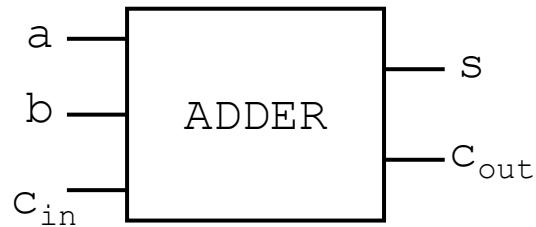


# VHDL and schematics

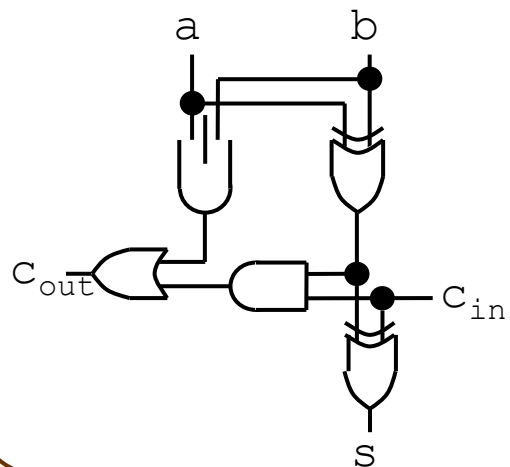
- The structure of a **VHDL module** is similar to that of a schematic.

## SCHEMATIC

### SYMBOL



### DESIGN



## VHDL MODULE

### ENTITY

Interface: input and output ports

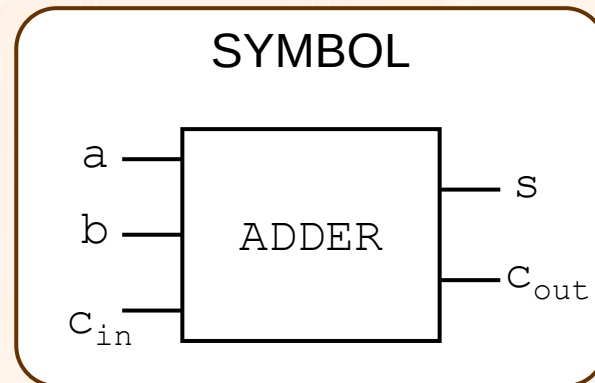
### ARCHITECTURE

Description of combinatorial and sequential logic; components and subprograms

# Entities

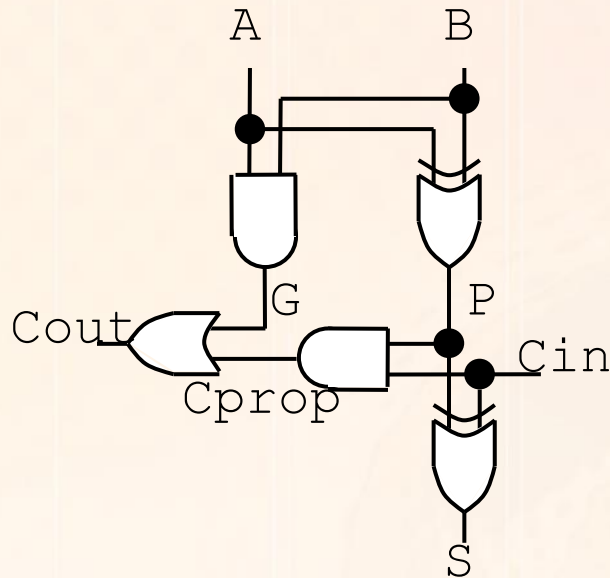
- An entity represents the *interface* of a circuit

```
entity full_adder is  
    port (A,B,Cin: in std_logic;  
          S, Cout: out std_logic);  
end full_adder;
```



# Architecture

- The *architecture* is the description of the circuit's functionality.



```
architecture arch of full_adder is
  signal P,G,Cprop: STD_LOGIC;
begin
    G <= A and B;
    P <= A xor B;
    Cprop <= P and Cin;
    Cout <= Cprop or G;
    S <= P xor Cin;
end arch;
```

- Note that, unlike schematics, the same circuit can be described in several different ways in VHDL



# Architecture

- *As in schematics, hierarchical design can be implemented using **components**.*

```
ARCHITECTURE structural OF top IS
    COMPONENT OR_gate IS
        PORT( a, b: IN STD_LOGIC;
              c: OUT STD_LOGIC);
    END COMPONENT;
    SIGNAL temp: STD_LOGIC;
BEGIN
    or1: OR_gate
        PORT MAP (a => a, b => b, c => temp);
    or2: OR_gate
        PORT MAP (a => temp, b => c, c => d);
END structural;
```

# VHDL - Overview

- *Limitations of VHDL:*
  - *Digital only, not analogue... yet*
  - *Somewhat cumbersome syntax (Verilog slightly better)*
  - *Dependent upon synthesis tools*
  - *Limited control on physical hardware layout (VLSI)*
- *Always remember:*
  - HDLs are not programming languages –if you “program” an HDL, you will generate very poor hardware (if at all). You must get out of the programming frame of mind and realize that you are describing a digital circuit.
  - HDLs are design entry tools – they will make entry and simulation more efficient, but they will not help you to actually design a circuit. GIGO (garbage in, garbage out) definitely applies!